

OpenLcbCLib

Quick Start Guide

Get an OpenLCB/LCC node running in minutes

ESP32 + Arduino IDE edition

OpenLcbCLib supports many platforms — this guide covers one specific path.

Table of Contents

1. What is OpenLCB/LCC?
2. What You Need
3. Project Setup for Arduino IDE
4. Choosing a Node ID
5. Starting a Project From Scratch
6. Understanding `openlcb_user_config.h`
7. Uploading and Running
8. What's Next

1. What is OpenLCB/LCC?

Note: This guide uses an ESP32 development board with Arduino IDE as a concrete, working example. OpenLcbCLib itself is platform-agnostic C — it can be adapted to any microcontroller with a C compiler. See the Developer Guide for other platforms and IDEs.

OpenLCB (Open Layout Control Bus) is an open-source project that produced a suite of specifications and technical notes for connecting model railroad devices. The NMRA adopted these specifications and technical notes as the LCC (Layout Command Control) standard. LCC and OpenLCB refer to the same protocol — LCC is simply the NMRA's official name for it.

Throttles, signal controllers, turnout drivers, occupancy detectors, and other accessories all talk to each other over a shared CAN bus (or WiFi) using a common language. OpenLcbCLib is a portable C library that implements these protocols for microcontrollers, handling all low-level protocol work so you can focus on what your node actually does.

Key Terms

- **Node:** Any device on the network. Your ESP32 board is a node.
- **Node ID:** A unique 48-bit number that identifies your node globally.
- **Event ID:** A 64-bit number representing something that happened or a command to perform.
- **Producer:** A node that sends an event when something happens.
- **Consumer:** A node that reacts to an event.
- **CDI:** Configuration Description Information — an XML file describing what settings your node exposes.
- **CAN Bus:** The physical wiring (two wires) that connects all nodes on the layout.

2. What You Need

Hardware

- ESP32 development board (any common 38-pin or 30-pin board)
- CAN bus transceiver module (SN65HVD230 or MCP2551 recommended)
- USB cable for programming
- LCC/OpenLCB layout bus, OR two ESP32 boards to test with

Note: The ESP32 has a built-in CAN controller called TWAI (Two-Wire Automotive Interface). You only need an external transceiver chip — you do NOT need a separate CAN controller.

CAN Transceiver Wiring

ESP32 Pin	Transceiver Pin
GPIO 21	TX (CAN transmit)
GPIO 22	RX (CAN receive)
3.3V or 5V	VCC (check your module datasheet)
GND	GND
(layout bus)	CANH and CANL to bus wires

Software

- Arduino IDE 2.x or PlatformIO (VSCode)
- ESP32 Arduino core installed in Arduino IDE (search “esp32” in Boards Manager)

- A modern web browser (Chrome, Firefox, or Edge) for the Node Wizard
- The OpenLcbCLib source code

3. Project Setup for Arduino IDE

The library is NOT installed as an Arduino Library. Instead, the source files live directly inside your sketch folder. This keeps every project self-contained with its own copy of the library.

Important Arduino IDE rule: Arduino IDE will only automatically compile C/C++ files that are in the sketch folder itself or in a subfolder named exactly src. Files in any other subfolder are ignored.

Starting From the ESP32 Example

The library includes a ready-to-run ESP32 example at:

OpenLcbCLib/applications/arduino/esp32/BasicNode/

1. Copy the entire BasicNode folder to your Arduino sketches folder.
2. Rename the folder to your project name, then rename the .ino file inside it to the same name. Arduino IDE requires the folder and .ino file to have exactly the same name.
3. Open the renamed .ino file in Arduino IDE.

Complete Project File Structure

BasicNode/	<- your sketch folder
BasicNode.ino	<- main sketch
openlcb_user_config.h	<- buffer sizes & feature flags
openlcb_user_config.c	<- node parameter data
src/	<- Arduino IDE compiles this subtree
application_callbacks/	<- your application logic
callbacks_can.h / .cpp	
callbacks_olcb.h / .cpp	
callbacks_config_mem.h / .cpp	
application_drivers/	<- hardware interface
esp32_can_drivers.h / .cpp	
esp32_drivers.h / .cpp	
openlcb_c_lib/	<- library (do not edit)
openlcb/	<- protocol engine
drivers/canbus/	<- CAN state machines
drivers/tcp_ip/	<- TCP/IP transport (compiled out for CAN nodes)
utilities/	<- helper files

Note: Never rename the src/ subfolder. Arduino IDE will stop compiling the library files if the folder has any other name.

4. Choosing a Node ID

Every node on a layout must have a unique 48-bit Node ID — think of it like a MAC address. If two nodes share an ID the network will have conflicts.

Option A: NMRA Member Range

If you are an NMRA member, you automatically have a personal Node ID range derived from your member number. Log in to the NMRA member portal to look up your assigned range.

Option B: Request a Manufacturer Range

If you are producing nodes commercially or distributing firmware publicly, request a dedicated range through the OpenLCB group. See the registry at registry.openlcb.org/uniqueidranges.

Option C: Development / Test Range

For personal testing on your own layout, you can use Node IDs from the reserved development range. These are safe to use privately but should not be used in nodes you distribute to others.

Setting the Node ID

BasicNode demo — the Node ID is defined as a constant at the top of the sketch:

```
// In BasicNode.ino, find and change this line:
#define NODE_ID 0x050101010777

// Use any value in the development range:
#define NODE_ID 0x050101010001 // node 1
#define NODE_ID 0x050101010002 // node 2
// Keep a list - every node needs a different ID
```

Wizard-generated projects — enter your Node ID in the Project Options panel before you generate the code. The Wizard accepts the ID in dotted hex format (xx.xx.xx.xx.xx.xx) and writes it into the generated files automatically.

Note: Every node on your layout must have a DIFFERENT Node ID, even when using the development range. Keep a list so you do not accidentally reuse one.

5. Starting a Project From Scratch

Once you are comfortable with the BasicNode demo, you will want to start your own project. Open the browser-based Node Wizard at `tools/node_wizard/node_wizard.html` in any modern browser — no internet connection required. Work through the sidebar from top to bottom:

Sidebar Section	What You Do There
Node Type	Pick the role of your node: Basic, Typical, Train, or Train Controller.
CDI	Describe the settings your node exposes to JMRI and other tools. Required for Typical, Train, and Train Controller nodes.
FDI	Describe the DCC functions your decoder supports. Train nodes only.
Target Platform	Select your target hardware. The Wizard pre-fills working driver code for the chosen platform.
Callbacks	Choose which protocol events your application needs to respond to.
Generated Files	Preview the output file tree and click Download ZIP to get your project.

Platform	Framework Layout
ESP32 + TWAI	Arduino / PlatformIO
ESP32 + WiFi GridConnect	Arduino / PlatformIO
RP2040 + MCP2517FD	Arduino (Earle Philhower core)
STM32F4xx + CAN	STM32 HAL (CubeIDE)
TI MSPM0 + MCAN	TI DriverLib (CCS / Theia)
None / Custom	Your own toolchain

Note: The Developer Guide covers the Node Wizard in full detail, including every section, the CDI editor, and the difference between Arduino and non-Arduino folder layouts.

6. Understanding openlcb_user_config.h

This is the most important file in your project. It tells the library how much memory to allocate and which features to compile in. Every value is mandatory.

```
// --- Transport -- exactly one must be defined -----
#define OPENLCB_COMPILE_CAN
// #define OPENLCB_COMPILE_TCP

// --- Feature Flags -----
// Uncomment to enable the protocols your node uses.
#define OPENLCB_COMPILE_EVENTS           // event producer/consumer
#define OPENLCB_COMPILE_DATAGRAMS       // needed for config memory
#define OPENLCB_COMPILE_MEMORY_CONFIGURATION // CDI/settings support
#define OPENLCB_COMPILE_FIRMWARE        // firmware upgrade (on in BasicNode)
// #define OPENLCB_COMPILE_BROADCAST_TIME // clock synchronization
// #define OPENLCB_COMPILE_TRAIN         // locomotive control
// #define OPENLCB_COMPILE_TRAIN_SEARCH  // throttle train discovery
// #define OPENLCB_COMPILE_STREAM        // stream transport
// #define OPENLCB_COMPILE_DCC_DETECTOR  // DCC detection protocol
// #define OPENLCB_COMPILE_DCC_CV       // DCC CV programming space 0xF8

// --- Buffer Pool -----
#define USER_DEFINED_BASIC_BUFFER_DEPTH 32 // 16 bytes each
#define USER_DEFINED_DATAGRAM_BUFFER_DEPTH 4 // 72 bytes each
#define USER_DEFINED_SNIP_BUFFER_DEPTH 4 // 256 bytes each
#define USER_DEFINED_STREAM_BUFFER_DEPTH 1 // stream data

// --- Node Count -----
#define USER_DEFINED_NODE_BUFFER_DEPTH 4

// --- Events -----
#define USER_DEFINED_PRODUCER_COUNT 64
#define USER_DEFINED_CONSUMER_COUNT 64
```

Note: The Node Wizard generates this file for you. The BasicNode example already includes it with sensible defaults for ESP32.

7. Uploading and Running

In Arduino IDE

1. Open BasicNode.ino in Arduino IDE.
2. From Tools, select your board: "ESP32 Dev Module".
3. Select the correct COM port.
4. BasicNode demo: change the #define NODE_ID at the top of the sketch to your chosen value (see Section 4).
Wizard-generated project: the Node ID was already set in Project Options — nothing to change here.
5. Click Upload.

What Happens at Power-On

When the node starts it goes through the LCC Alias Negotiation process. After a few seconds it becomes active and is visible to JMRI and other LCC tools on the bus.

Testing With JMRI

- Open JMRI and connect to your LCC network.
- Go to the LCC menu and choose Configure Nodes. Your node should appear in the list.
- Select the node and click Open Configuration Dialog. JMRI reads the CDI and shows your settings.
- You can change the node name and description from here.

Note: The BasicNode demo does not implement real persistent storage. Changes you make to the node name and description through JMRI will not survive a power cycle. How you store configuration data is up to you — common choices include external EEPROM, an SD card, or the ESP32's internal flash (NVS/Preferences). The Developer Guide covers how to wire your storage choice into the config memory callbacks.

8. What's Next

You now have a working OpenLCB/LCC node. Here are the typical next steps:

Add Your Own Producer Events

```
// In CallbacksOlc_initialize():
openlcb_node_t *node = OpenLcbNode_get_first(0);
OpenLcbApplication_register_producer_eventid(
    node, 0x0501010107770001, EVENT_STATUS_CLEAR);

// In CallbacksOlc_on_100ms_timer():
if (button_is_pressed())
    OpenLcbApplication_send_event_pc_report(
        node, 0x0501010107770001);
```

React to Consumer Events

```
// Register the event to listen for:
OpenLcbApplication_register_consumer_eventid(
    node, 0x0501010107770002, EVENT_STATUS_UNKNOWN);

// In your on_consumed_event_pcer callback:
void my_event_handler(openlcb_node_t *node,
    uint16_t index, event_id_t *event_id,
    event_payload_t *payload)
{
    if (*event_id == 0x0501010107770002)
        turn_on_led();
}
```

Read the Developer Guide

The companion document, OpenLcbCLib Developer Guide, walks through every part of the system in detail: the Node Wizard, drivers, callbacks, CDI configuration, and more.

Online Help

Full documentation: <https://jimkueneman.github.io/OpenLcbCLib/documentation/help/html/>